

[CS230] Optimization Algorithms: Learning Rate Decay

Lecturer: Gemini (Integrated Editor)

□ Course Table of Contents

Chapter 1-4. Neural Networks Basics - *Completed*

Chapter 5. Practical Aspects of Deep Learning (Current Unit)

- 5.1-5.5 Regularization & Data Setup - *Completed*
- 5.6 Mini-batch Gradient Descent - *Completed*
- 5.7-5.8 Momentum, RMSprop, Adam - *Completed*
- **5.9 Learning Rate Decay**
 - * The Parking Problem (Oscillation)
 - * Decay Schedules (Inverse Time, Exponential, Step)
 - * Implementation & Visualization
- 5.10 Hyperparameter Tuning Strategy - *Upcoming*

지난 시간 복습 및 연결

지난 시간에 우리는 Adam Optimizer를 통해 최적화의 정점에 도달했습니다. 하지만 아주 미세한 문제가 하나 남았습니다. 학습 후반부가 되면 손실 함수(Cost)의 최저점 근처에서 모델이 안착하지 못하고 계속 맴도는 진동(**Oscillation**) 현상이 발생합니다. 주차장에서 시속 100km로 달리면 절대 주차 칸에 차를 넣을 수 없습니다. 목적지 근처에서는 속도를 줄여야 합니다. 이것이 바로 학습률 감쇠(**Learning Rate Decay**)가 필요한 이유입니다.

1 Unit Overview

□ 핵심 목표

이 단원은 학습 단계별로 학습률(α)을 조절하여 모델 성능을 극대화하는 기법을 다룹니다.

- **이유:** 고정된 학습률이 최저점 근처에서 수렴하지 못하는 이유를 기하학적으로 이해합니다.
- **전략:** 시간 기반(Inverse Time), 지수(Exponential), 계단식(Step) 감쇠의 수식적 차이를 파악합니다.
- **구현:** Python으로 스케줄러를 구현하고 에폭(Epoch)에 따른 변화를 그래프로 확인합니다.

2 Essential Terminology

용어	기호	설명
Learning Rate	α	한 번 업데이트할 때 이동하는 보폭의 크기.
Decay Rate	k	학습률을 얼마나 빨리 줄일지 결정하는 계수.
Epoch	t	전체 데이터를 한 번 학습한 횟수. (시간 단위)
Initial LR	α_0	학습 시작 시점의 초기 학습률.

3 Core Concepts: 속도 조절의 미학

3.1 1. The Parking Problem (왜 줄여야 하는가?)

□ 고속도로와 주차장 비유 (직관적 비유)

- **Early Stage (고속도로):** 최저점이 멀리 있습니다. 이때는 보폭이 커야(High α) 빨리 접근할 수 있습니다.
- **Late Stage (주차장):** 최저점 근처입니다. 이때도 보폭이 크다면 구멍을 지나쳐 버리고(Overshooting), 다시 돌아오려다 또 지나칩니다.
- **Solution:** 목적지에 가까워질수록 속도를 서서히 줄여서 정밀하게 주차(수렴)해야 합니다.

3.2 2. Decay Schedules (감쇠 전략)

시간(t , Epoch)이 지날수록 α 를 줄이는 대표적인 공식들입니다.

- **1. Inverse Time Decay (시간 기반):**

$$\alpha = \frac{1}{1 + k \cdot t} \alpha_0$$

가장 완만하게 줄어듭니다.

- **2. Exponential Decay (지수 감쇠):**

$$\alpha = k^t \cdot \alpha_0 \quad (k < 1, \text{예: } 0.95)$$

빠르게 0으로 수렴합니다.

- **3. Step Decay (계단식 감쇠):** 10 에폭마다 절반으로 똑 떨어뜨립니다. (ResNet 등 심층 모델에서 선호)

4 Deep Dive: Adaptive Methods와의 관계

”교수님, Adam이 알아서 학습률 조절해주지 않나요?”

- **Adam/RMSprop:** 파라미터마다 *개별적으로* 학습률을 조절(Adaptive)하지만, 전체적인 *글로벌 학습률*(α) 자체는 고정되어 있습니다.
- **결론:** Adam을 쓰더라도 Learning Rate Decay를 함께 적용하면, 최저점에서의 진동을 줄여 성능을 더 높일 수 있습니다. (SOTA 모델들의 필수 테크닉)

5 Implementation: Decay Scheduler

직접 스케줄러를 만들고 그래프를 그려봅시다.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 class LRScheduler:
5     def __init__(self, init_lr=1.0):
6         self.init_lr = init_lr
7
8     def inverse_time_decay(self, epoch, k=0.1):
9         """lr = lr0 / (1 + kt)"""
10        return self.init_lr / (1 + k * epoch)
11
12    def exponential_decay(self, epoch, k=0.95):
13        """lr = lr0 * k^t"""
14        return self.init_lr * np.power(k, epoch)
15
16    def step_decay(self, epoch, drop=0.5, interval=10):
17        """특정 간격(interval)마다 drop 비율만큼 감소"""
18        exponent = np.floor((1 + epoch) / interval)
19        return self.init_lr * np.power(drop, exponent)
20
21 # --- 시각화 ---
22 if __name__ == "__main__":
23     epochs = np.arange(0, 100)
24     scheduler = LRScheduler(init_lr=1.0)
25
26     lr1 = [scheduler.inverse_time_decay(e) for e in epochs]
27     lr2 = [scheduler.exponential_decay(e) for e in epochs]
28     lr3 = [scheduler.step_decay(e) for e in epochs]
29
30     plt.figure(figsize=(10, 6))
31     plt.plot(epochs, lr1, label='Inverse Time')
32     plt.plot(epochs, lr2, label='Exponential')
33     plt.plot(epochs, lr3, label='Step Decay')
34     plt.title('Learning Rate Decay Schedules')
35     plt.xlabel('Epochs')
36     plt.ylabel('Learning Rate')
37     plt.legend()
38     plt.grid(True)
39     plt.show()
```

Listing 1: Learning Rate Schedulers

6 FAQ & Pitfalls

ReduceLROnPlateau (실전 꿀팁) 가장 실용적인 방법은 수식보다는 성능을 보고 줄이는 것입니다. ”지난 10 에폭 동안 Dev Error가 줄어들지 않았네? (Plateau)” → ”이제 정말 타격할 때다. 학습률을 1/10로 줄여라.” Keras나 PyTorch에서 ‘ReduceLROnPlateau’ 콜백을 사용하면 됩니다.

Q. k (감쇠율)가 너무 크면 어떻게 되나요?

A. 학습률이 너무 빨리 0이 되어버립니다. 최저점에 도달하기도 전에 모델이 멈춰버리는 조기 수렴(Premature Convergence) 문제가 발생합니다.

다음 단계 (Next Step)

이제 최적화도구들은 모두 갖췄습니다. 하지만 하이퍼파라미터가 너무 많아졌습니다. ($\alpha, \beta_1, \beta_2, \epsilon, k, \lambda, \text{batch_size}$)

이 많은 다이얼을 어떤 순서로, 어떻게 맞춰야 할까요? 사람이 일일이 돌려보기엔 시간이 너무 부족합니다. 다음 시간에는 [Hyperparameter Tuning] 전략을 통해, 이 복잡한 퍼즐을 체계적으로 푸는 법을 배웁니다. **Grid Search**와 **Random Search**의 승부가 펼쳐집니다.

□ 단원 요약 (Cheat Sheet)

1. **Need for Decay:** 고정 학습률은 최저점 근처에서 진동한다. 정밀한 수렴을 위해 줄여야 한다.
2. **Schedules:** Inverse Time(완만), Exponential(급격), Step(계단식) 등이 있다.
3. **Best Practice:** Dev Error가 정체될 때 줄이는 ReduceLROnPlateau 방식이 가장 효과적이다.
4. **With Adam:** Adam을 쓰더라도 Decay를 함께 쓰면 성능이 더 좋아진다.